ЛАБОРАТОРНА РОБОТА № 5

Тема: Пряма адресація та арифметика покажчиків, емуляція апаратних регістрів мікроконтролера, передача параметрів за адресою та вирішення конфліктів злиття (Merge Conflicts) у Git

МЕТА РОБОТИ:На практиці опанувати пряму роботу з оперативною пам'яттю комп'ютера за допомогою покажчиків та реалізувати механізм передачі параметрів за адресою (Passby-Address). Опанувати швидкісну адресну навігацію по суцільних масивах даних за допомогою арифметики покажчиків (ptr++ , ptr + i , різниця покажчиків ptrdiff_t) без застосування оператора індексації у квадратних дужках. Дослідити модифікацію апаратних регістрів «на місці» через покажчики з використанням чотирьох канонів бітових масок (встановлення, скидання, інвертування, тестування). Засвоїти інженерні стандарти захисту пам'яті за допомогою кваліфікатора const та перевірок на нульовий покажчик (NULL). Опанувати ключову навичку командної розробки в Git: навмисне створення, аналіз маркерів колізій та ручне вирішення конфлікту злиття (Merge Conflict). Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.F7.11.000 –Лр.5		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Іванюченко І.Є.			Звіт з лабораторної роботи №5	Літ.	Арк. 1 / Акрушів 8
Перевір.		Личенко М.С.					
Реценз.						ФІКТ, гр. КІ-26-1	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М.					

Варіант 11:

Таблиця 1. Варіанти до Завдання 1 (Вихідні параметри через покажчики

11	Вхід: катети прямокутного трикутника A і B. Вихідні параметри: покажчик на гіпотенузу C та покажчик на периметр P.

Таблиця 2. Варіанти до Завдання 2 (Модифікація 32-бітного регістра через покажчик)

11	Встановити в 1 біт 6; скинути в 0 біт 0; інвертувати біт 15; перевірити стан біта 24.

Таблиця 3. Варіанти до Завдання 3 (Адресна арифметика над буфером без [])

11	Перевірити, чи є масив симетричним (паліндромом), рухаючи два покажчики від протилежних кінців буфера до центральної адреси.

Опис створення та вирішення конфлікту Git.

```
<<<<<<< HEAD
Версія налаштувань контролера: Базова конфігурація периферії
=======
Версія налаштувань контролера: Пріоритетна конфігурація LAB05
>>>>>>> lab05
```

Спочатку в основній гілці main було створено файл conflict_test.txt із початковою конфігурацією контролера. Після цього було виконано перехід до гілки lab05, у якій внесено зміни до того самого файлу та створено окремий коміт.

Пізніше було здійснено перехід назад до гілки main і виконано команду злиття git merge lab05. Оскільки в обох гілках один і той самий файл було змінено по-різному, Git виявив конфлікт злиття.

Для його вирішення файл conflict_test.txt було відкрито та відредаговано вручну. Конфліктні позначки Git було видалено, а замість двох різних варіантів залишено узгоджений текст: «Версія налаштувань контролера: Узгоджена фінальна конфігурація».

Після цього файл було додано до індексу Git за допомогою команди git add, а результат вирішення конфлікту зафіксовано окремим комітом. Перевірка історії командою git log --graph --oneline --all підтвердила успішне злиття гілок.

Етап 1. Вихідні параметри та передача за адресою (task1.c)

Лістинг:

```c

#include <stdio.h>
#include <math.h>

int calculate_triangle(const double *a, const double *b,
                       double *c, double *p)
{
    if (a == NULL || b == NULL || c == NULL || p == NULL)
    {
        return 1;
    }

    if (!isfinite(*a) || !isfinite(*b) || *a <= 0 || *b <= 0)
    {
        return 1;
    }

    *c = sqrt((*a) * (*a) + (*b) * (*b));
    *p = *a + *b + *c;

    return 0;
}

int main(void)
{
    double a = 3.0;
    double b = 4.0;

    double c = 0.0;
    double p = 0.0;

    int result;

    result = calculate_triangle(&a, &b, &c, &p);

    if (result == 0)
    {
        printf("Cathetus A: %.2f\n", a);
        printf("Cathetus B: %.2f\n", b);
        printf("Hypotenuse C: %.2f\n", c);
        printf("Perimeter P: %.2f\n", p);
    }
    else
    {
        printf("Error: incorrect input data!\n");
    }

    return 0;
}
```

Перевірка:

```
Cathetus A: 3.00
Cathetus B: 4.00
Hypotenuse C: 5.00
Perimeter P: 12.00
```

Пояснення:

У першому завданні необхідно розробити програму для обчислення гіпотенузи та периметра прямокутного трикутника за двома заданими катетами A і B.

На початку програми підключаються бібліотеки stdio.h для роботи з введенням і виведенням даних та math.h для використання математичної функції sqrt().

Основна частина програми містить функцію calculate_triangle(), яка приймає адреси змінних за допомогою вказівників. Спочатку функція перевіряє, чи не є передані вказівники нульовими, а також чи є значення катетів додатними та коректними.

Далі виконується обчислення гіпотенузи за теоремою Піфагора: $C = \sqrt{(A^2 + B^2)}$. Після цього визначається периметр трикутника як сума двох катетів і гіпотенузи: $P = A + B + C$.

Отримані результати записуються у змінні c та p через передані вказівники. Функція повертає 0 у разі успішного виконання або 1, якщо виявлено некоректні дані.

У функції main() задаються початкові значення катетів, викликається функція обчислення та виводяться результати на екран. Таким чином, програма демонструє використання вказівників для передачі даних і повернення результатів із функції.

Етап 2. Модифікація 32-бітного регістра «на місці» через покажчик (task2.c)

Лістинг:

```c
#include <stdio.h>
#include <stdint.h>

void configure_register(uint32_t *reg)
{
    if (reg == NULL)
    {
        return;
    }

    *reg |= (1u << 6);

    *reg &= ~(1u << 0);

    *reg ^= (1u << 15);
}

int check_status_flag(const uint32_t *reg, uint8_t bit_pos)
{
    if (reg == NULL || bit_pos >= 32)
    {
        return 0;
    }

    return ((*reg >> bit_pos) & 1u);
}

int main(void)
{
    uint32_t reg = 0x01010001u;

    printf("Register before: 0x%08X\n", reg);

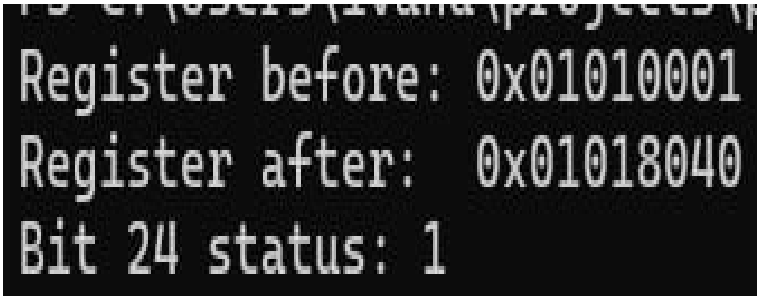
    configure_register(&reg);

    printf("Register after:  0x%08X\n", reg);

    printf("Bit 24 status: %d\n",
           check_status_flag(&reg, 24));

    return 0;
}
```

Перевірка:



Пояснення:

У другому завданні необхідно розробити програму для роботи з апаратним регістром за допомогою вказівників та бітових операцій.

На початку програми підключаються бібліотеки stdio.h для введення та виведення даних і stdint.h для використання типу uint32_t, який зберігає 32-бітне беззнакове число.

Основна функція configure_register() отримує адресу регістра через вказівник. За допомогою бітових операцій вона встановлює біт №6, очищує біт №0 та змінює стан біта №15 на протилежний. Для цього використовуються операції OR (|), AND (&) та XOR (^).

Функція check_status_flag() перевіряє стан заданого біта регістра. Вона приймає адресу регістра та номер біта, після чого визначає, чи встановлений він у значення 1. Також передбачена перевірка коректності вказівника та номера біта.

У функції main() задається початкове значення регістра 0x01010001, після чого виводиться його початковий стан. Далі викликається функція налаштування регістра, і на екран виводиться результат змін. Наприкінці перевіряється стан біта №24.

Таким чином, програма демонструє принцип роботи з бітовими масками, зміну окремих бітів та перевірку стану апаратного регістра.

Етап 3. Обробка масиву через адресну арифметику (task3.c)

Лістинг:

```c
#include <stdio.h>
#include <stddef.h>

int is_palindrome(const int *start_ptr, ptrdiff_t length)
{
    if (start_ptr == NULL || length <= 0)
    {
        return 0;
    }

    const int *left_ptr = start_ptr;
    const int *right_ptr = start_ptr + length - 1;

    while (left_ptr < right_ptr)
    {
        if (*left_ptr != *right_ptr)
        {
            return 0;
        }

        left_ptr++;
        right_ptr--;
    }

    return 1;
}

int main(void)
{
    int numbers[] = {1, 2, 3, 2, 1};

    const int *start_ptr = numbers;
    const int *end_ptr = numbers + sizeof(numbers) / sizeof(*numbers);

    ptrdiff_t length = end_ptr - start_ptr;

    printf("Array: ");

    const int *ptr = start_ptr;

    while (ptr < end_ptr)
    {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\n");

    if (is_palindrome(start_ptr, length))
    {
        printf("Result: The array is a palindrome.\n");
    }
    else
    {
        printf("Result: The array is not a palindrome.\n");
    }

    return 0;
}
```

Перевірка:

```
Array: 1 2 3 2 1
Result: The array is a palindrome.
```

Пояснення:

У третьому завданні необхідно розробити програму для перевірки масиву цілих чисел на наявність паліндрому. Паліндромом називається послідовність, яка однаково читається зліва направо та справа наліво.

Для виконання завдання використовується функція is_palindrome(), яка приймає вказівник на початок масиву та його довжину. За допомогою двох вказівників left_ptr і right_ptr програма порівнює елементи з протилежних кінців масиву.

Якщо значення елементів відрізняються, функція повертає 0, що означає, що масив не є паліндромом. Якщо всі відповідні елементи збігаються, функція повертає 1. Порівняння продовжується до моменту, поки вказівники не зустрінуться або не перетнуться.

У функції main() створюється масив {1, 2, 3, 2, 1}, визначається його довжина за допомогою вказівників та виводяться елементи на екран. Після цього викликається функція перевірки, яка визначає, чи є масив паліндромом.

Таким чином, програма демонструє використання арифметики вказівників для обходу масиву та порівняння його елементів без використання індексів.

Висновки:

Під час виконання лабораторної роботи було закріплено навички роботи з вказівниками в мові програмування С, а також вивчено способи передачі даних через адреси змінних. Було реалізовано обчислення параметрів прямокутного трикутника, роботу з бітовими операціями та перевірку масиву на паліндром за допомогою арифметики вказівників.

Також було отримано практичний досвід роботи з Git: створення гілок, виконання комітів, злиття змін та вирішення конфліктів. У результаті всі поставлені завдання було виконано, а отримані знання допомогли краще зрозуміти роботу з пам’яттю та інструментами контролю версій.